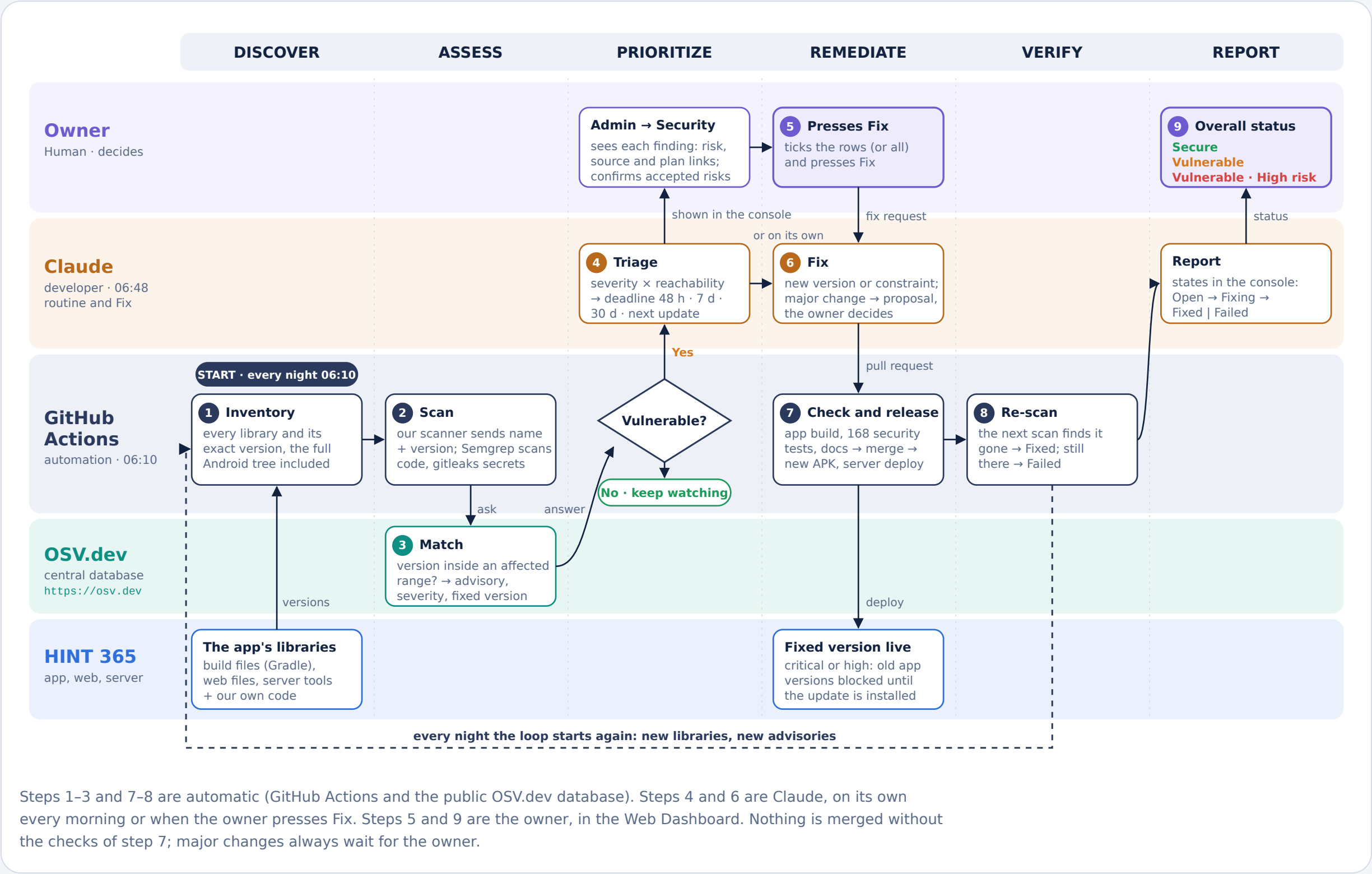


How a vulnerability is found and fixed

Read the drawing left to right: the six phases of the standard vulnerability management lifecycle. Each row is who does the step. It starts every night at 06:10 on its own; the owner steps in to see the risk, press Fix, and decide the larger changes.

🕒 numbered steps, in order ♦ a decision - - - the loop back to the start, every night



Steps 1–3 and 7–8 are automatic (GitHub Actions and the public OSV.dev database). Steps 4 and 6 are Claude, on its own every morning or when the owner presses Fix. Steps 5 and 9 are the owner, in the Web Dashboard. Nothing is merged without the checks of step 7; major changes always wait for the owner.

CENTRAL DATABASE

Where vulnerabilities are known

OSV.dev (osv.dev), the open vulnerability database of Google and the open-source community, queried through api.osv.dev. It gathers the advisories of the [GitHub Advisory Database](#), [CVE](#) (scored by [NIST NVD](#)), Android, Maven and npm.

THE RULE

When a library is vulnerable

Every advisory names a library and the **range of versions** it affects. A library is vulnerable when the **exact version we use falls inside that range**. Whether it matters for us is decided at step 4: shipped to phones, web or server, or only used to build.

DEADLINES

How fast it is fixed

CRITICAL	48 hours · old app versions blocked
HIGH	7 days · old app versions blocked
MODERATE	30 days
LOW	next update

THE CONSOLE

What the owner sees in Admin → Security

On top, the overall status: **SECURE** nothing open, **VULNERABLE** something open, **VULNERABLE · HIGH RISK** a high or critical finding open. Below, one row per finding: library, version, where it sits, risk, a link to the advisory and a link to the plan. Tick the rows, press **Fix**, and each row moves by itself from **Open** to **Fixing**, then **Fixed** or **Failed**.

COST

0 \$ extra

GitHub Actions free plan, Dependabot, Semgrep and gitleaks (open source), OSV.dev (public), Claude on the owner's plan.

TODAY

Full scan, 28 September 2026

WHERE	LIBRARIES	ADVISORIES	OUR RATING	PLAN
In the app (shipped to phones)	Android tree, web, server	0	nothing open	—
Android build tools (not in the app)	build plugin and what it pulls in: netty, Bouncy Castle, protobuf...	58	3 HIGH 22 MODERATE 33 LOW	update the Android build plugin (PR #94): the owner decides
Kotlin build plugin	kotlin-gradle-plugin 1.9.24	1	LOW	issue #97
Our code (Semgrep)	manifest: launcher screen exported	1	LOW false positive	required by Android

365 libraries checked, 272 of them Android (full tree). gitleaks found no secret in the history; the old signing-key password in build.gradle.kts is known (test report F-01) and goes away when the owner moves the key to GitHub secrets.

TOOLS

What runs

- Our scanner `vuIn-scan.mjs` + OSV.dev
- [Semgrep](#) on our code
- [gitleaks](#) on secrets
- Dependabot, every Monday